Slide 1 de 30

Aula 4 — Herança, Polimorfismo, static e Prototype

A aula mais densa do módulo

Hoje é a aula com mais conceitos novos do módulo. Vamos ver 4 pilares que completam a base de OOP: **herança**, **polimorfismo**, **static** e **prototype**.

Aula 1	Aula 2	Aula 3	Aula 4 — Hoje
Classes e atributos	Métodos	Encapsulamento	Herança e polimorfismo
O molde	As ações	A proteção	O reaproveitamento
Produto TEM preço	Produto SABE calcular	Preço é PROTEGIDO	ProdutoFisico HERDA de Produto

---

Slide 2 de 30

O Problema: Código Repetido

Quando classes parecidas geram duplicação

Olhem essas duas classes. O que vocês percebem?

Slide 3 de 30

A Solução: Herança

Extrair o que é comum para uma classe pai

Herança permite criar uma classe base com tudo que é **comum** e classes filhas que **herdam** tudo e adicionam o que é **específico**.

O código de `comer()`, `dormir()` e o constructor existem **uma única vez** em `Animal`. Cada filho herda tudo automaticamente e só adiciona o que é diferente.

Slide 4 de 30

extends e super — A Sintaxe da Herança

Como implementar herança em JavaScript

Slide 5 de 30

Herança — Testando

Vendo a herança em ação

```
const rex = new Cachorro("Rex", 3, "Pastor Alemão");

// Métodos HERDADOS de Animal — não precisou reescrever!
rex.comer();   // "Rex está comendo."
rex.dormir();  // "Rex está dormindo."

// Método SOBRESCRITO — executa a versão do Cachorro
rex.falar();   // "Rex diz: Au au!" (não "Rex faz um som.")

// Método PRÓPRIO do Cachorro
rex.buscar();  // "Rex buscou a bolinha!"

// Atributos: herdados + próprios
console.log(rex.nome);  // "Rex" (herdado)
console.log(rex.idade); // 3 (herdado)
console.log(rex.raca);  // "Pastor Alemão" (próprio)

// instanceof funciona com a hierarquia
console.log(rex instanceof Cachorro); // true
console.log(rex instanceof Animal);   // true ← é ambos!
```

Slide 6 de 30

O Que Acontece Sem super()?

Um erro que vocês VÃO encontrar

```
class Gato extends Animal {
  constructor(nome, idade, indoor) {
    // ❌ Esqueceu o super()!
    this.indoor = indoor;
  }
}

const mimi = new Gato("Mimi", 2, true);
// ❌ ReferenceError: Must call super constructor in
derived class
//    before accessing 'this' or returning from
derived constructor
```

```
// ✅ Correto:
class Gato extends Animal {
  constructor(nome, idade, indoor = true) {
    super(nome, idade);   // PRIMEIRO: inicializa a parte Animal
    this.indoor = indoor; // DEPOIS: inicializa a parte Gato
  }

  falar() {
    console.log(`${this.nome} diz: Miau!`);
  }
}
```

Slide 7 de 30

Verificação — extends e super

Testando os fundamentos de herança

O que acontece se o constructor da classe filha NÃO chamar `super()`?

- O constructor do pai é chamado automaticamente
- ✓ ReferenceError — obrigatório chamar super() antes de usar this (Resposta Correta)
- Os atributos do pai ficam undefined
- Funciona normalmente, mas sem herança

Explicação: Correto! O JavaScript exige que `super()` seja chamado na primeira linha do constructor da classe filha. Sem ele, não é possível usar `this` e ocorre um ReferenceError.

Slide 8 de 30

Caso de Uso Real — Sistema de Pagamentos

Herança em um cenário profissional

Em qualquer e-commerce existem vários tipos de pagamento: cartão, PIX, boleto. Todos compartilham dados base (valor, data, status), mas cada um tem lógica específica.

Slide 9 de 30

Sistema de Pagamentos — Implementação

Classe pai e filhas com super.metodo()

Slide 10 de 30

Sistema de Pagamentos — Testando

Cada tipo processa de forma diferente

```
const cartao = new PagamentoCartao(300, "Visa", 3);
cartao.processar();
// "Pagamento de R$300.00 processado."  ← super.processar()
// "Cartão Visa em 3x de R$100.00"      ← lógica específica

const pix = new PagamentoPix(150, "email@email.com");
pix.processar();
// "Pagamento de R$150.00 processado."  ← super.processar()
// "PIX enviado para: email@email.com"  ← lógica específica

// Todos são Pagamento!
console.log(cartao instanceof Pagamento);      // true
console.log(pix instanceof Pagamento);          // true
console.log(cartao instanceof PagamentoCartao); // true
```

O `super.processar()` executa a lógica **comum** (marcar como processado, log genérico). Depois, cada classe filha adiciona sua lógica **específica** (parcelas do cartão, chave do PIX). Isso evita duplicar código.

Slide 11 de 30

O que é Polimorfismo?

Mesmo método, comportamentos diferentes

Polimorfismo vem do grego: "poli" = muitas, "morfo" = formas. É a capacidade de um **mesmo método** ter **comportamentos diferentes** dependendo da classe que o implementa.

```
// Array com tipos MISTURADOS de pagamento
const pagamentos = [
  new PagamentoCartao(100, "Visa", 3),
  new PagamentoPix(50, "email@email.com"),
  new PagamentoCartao(200, "Master", 1),
  new PagamentoPix(75, "11999998888"),
];

// Polimorfismo em ação:
// O MESMO método, comportamento DIFERENTE em cada tipo
pagamentos.forEach(p => p.processar());
// "Pagamento de R$100.00 processado."
// "Cartão Visa em 3x de R$33.33"
// "Pagamento de R$50.00 processado."
// "PIX enviado para: email@email.com"
// "Pagamento de R$200.00 processado."
// "Cartão Master em 1x de R$200.00"
// "Pagamento de R$75.00 processado."
// "PIX enviado para: 11999998888"
```

Slide 12 de 30

Verificação — Polimorfismo

Testando o conceito

No código `pagamentos.forEach(p => p.processar())`, por que funciona mesmo com tipos diferentes no array?

Porque o JavaScript converte todos para o mesmo tipo

✓ Porque todos herdam de Pagamento e têm seu próprio processar() — cada um executa a SUA versão (Resposta Correta)

Porque o forEach sabe diferenciar os tipos

Porque processar() é um método estático

Explicação: Exato! Todos herdam de `Pagamento` e possuem o método `processar()`. Cada classe sobrescreve com sua lógica específica. O forEach não precisa saber o tipo — só chama o método e cada um faz o seu.

Slide 13 de 30

Herança nas Formas Geométricas

Aplicando herança no estudo de caso que acompanha o módulo

Vamos retomar nossas formas geométricas e aplicar herança. Uma classe base `Forma` concentra o que é comum.

```
class Forma {
  constructor(nome = "Forma", cor = "preto") {
    this.nome = nome;
    this.cor = cor;
  }

  calcularArea() {
    return 0; // Será sobrescrito pelas filhas
  }

  calcularPerimetro() {
    return 0; // Será sobrescrito pelas filhas
  }

  exibirInfo() {
    console.log(`${this.nome} (${this.cor}):`);
    console.log(`  Área: ${this.calcularArea().toFixed(2)}`);
    console.log(`  Perímetro:
${this.calcularPerimetro().toFixed(2)}`);
  }
}

class Circulo extends Forma {
  #raio;
  constructor(raio = 1, cor = "preto") {
    super("Círculo", cor);
    this.#raio = raio;
  }
  get raio() { return this.#raio; }
  calcularArea() { return Math.PI * this.#raio ** 2; }
  calcularPerimetro() { return 2 * Math.PI * this.#raio; }
}
```

Slide 14 de 30

Polimorfismo nas Formas

exibirInfo() funciona para QUALQUER forma

```
class Retangulo extends Forma {
  constructor(base = 1, altura = 1, cor = "preto") {
    super("Retângulo", cor);
    this.base = base;
    this.altura = altura;
  }
  calcularArea() { return this.base * this.altura; }
  calcularPerimetro() { return 2 * (this.base + this.altura); }
  ehQuadrado() { return this.base === this.altura; }
}

// Polimorfismo: exibirInfo() do PAI chama calcularArea()
de cada FILHO
const formas = [
  new Circulo(5, "azul"),
  new Retangulo(4, 6, "vermelho"),
  new Circulo(3, "verde"),
];

formas.forEach(f => f.exibirInfo());
// Círculo (azul):
//   Área: 78.54
//   Perímetro: 31.42
// Retângulo (vermelho):
//   Área: 24.00
//   Perímetro: 20.00
// Círculo (verde):
//   Área: 28.27
//   Perímetro: 18.85
```

Slide 15 de 30

Verificação — Herança e Polimorfismo

Juntando os conceitos

No código `formas.forEach(f => f.exibirInfo())`, como o JavaScript sabe qual `calcularArea()` chamar para cada forma?

Usa sempre o calcularArea() da classe Forma

O JavaScript verifica o tipo e escolhe: se for Circulo, usa $\pi \times r^2$; se for Retangulo, usa b×h

✓ Cada instância carrega a referência para o método da SUA classe — o this aponta para o objeto correto (Resposta Correta)

O forEach converte todas para a mesma classe

Explicação: Correto! Cada instância sabe de qual classe veio. Quando `this.calcularArea()` é chamado em `exibirInfo()`, o `this` aponta para a instância (Circulo ou Retangulo), então o JS busca o método na classe correta.

Slide 16 de 30

Variáveis e Métodos Estáticos — static

O que pertence à CLASSE, não à instância

Até agora, tudo que vimos pertence à **instância** — cada objeto tem seus próprios atributos e métodos. Com `static`, criamos membros que pertencem à **classe em si**, compartilhados entre todos os objetos.

Slide 17 de 30

static na Prática — Contador de IDs

O caso de uso mais clássico

```
const p1 = new Pedido("Ana", ["Pizza"]);
const p2 = new Pedido("Carlos", ["Hambúrguer", "Suco"]);
const p3 = new Pedido("Bia", ["Salada"]);

console.log(p1.numero); // 1
console.log(p2.numero); // 2
console.log(p3.numero); // 3

// Método estático — chamado na CLASSE, não na instância
console.log(Pedido.totalPedidos()); // 3

// ❌ Não funciona na instância:
// p1.totalPedidos(); // TypeError!
```

Slide 18 de 30

Vocês Já Usam static Sem Saber!

Exemplos do JavaScript nativo

Código	O que é
`Math.random()`	Método estático de Math
`Math.PI`	Propriedade estática de Math
`Math.floor(4.7)`	Método estático de Math
`Date.now()`	Método estático de Date
`Number.isInteger(42)`	Método estático de Number
`Array.isArray([1,2])`	Método estático de Array
`JSON.stringify(obj)`	Método estático de JSON
`Object.keys(obj)`	Método estático de Object

Slide 19 de 30

Verificação — static

Testando o conceito

Por que `static` NÃO pode usar `this` para acessar atributos da instância?

É um bug do JavaScript

✓ Porque static pertence à CLASSE, e pode ser chamado sem criar instância — não existe instância para o this apontar (Resposta Correta)

Porque static é mais antigo que this

Na verdade, pode sim usar this normalmente

Explicação: Exato! Um método estático pertence à classe, não a uma instância. Como `Pedido.totalPedidos()` é chamado na classe (sem `new`), não existe instância — logo `this` não aponta para nenhum objeto.

Slide 20 de 30

Prototype — O Motor por Baixo do Capô

Como a herança realmente funciona em JavaScript

Todo objeto em JavaScript tem uma **cadeia de prototypes** — é como a herança funciona por baixo dos panos. A sintaxe `class` que usamos é na verdade **açúcar sintático** (syntactic sugar) sobre esse sistema de prototypes.

Quando chamamos `rex.comer()`, o JS procura `comer` no próprio `rex`. Não encontra? Sobe para `Cachorro.prototype`. Não encontra? Sobe para `Animal.prototype`. Encontrou! Executa.

```
// Os métodos vivem no prototype, não no objeto!
console.log(typeof Cachorro.prototype.buscar);    // "function"
console.log(typeof Animal.prototype.comer);       // "function"

// Todos os arrays herdam de Array.prototype
console.log(typeof Array.prototype.push);         // "function"
console.log(typeof Array.prototype.filter);       // "function"
```

Slide 21 de 30

Estendendo Classes Nativas com Prototype

Adicionando métodos a Array, String e Math

Como os métodos vivem no `prototype`, podemos **adicionar** novos métodos a classes que já existem — inclusive as nativas do JavaScript!

```
// Adicionando método ao Array
Array.prototype.ultimo = function() {
  return this[this.length - 1];
};

const frutas = ["maçã", "banana", "uva"];
console.log(frutas.ultimo()); // "uva" ✅

const nums = [10, 20, 30];
console.log(nums.ultimo());   // 30 ✅
```

```
// Adicionando método ao String
String.prototype.capitalizar = function() {
  return this.charAt(0).toUpperCase()
+ this.slice(1).toLowerCase();
};

console.log("jOÃO".capitalizar());       // "João"
console.log("maria".capitalizar());      // "Maria"
console.log("ANA SILVA".capitalizar());  // "Ana silva"
```

```
// Math não é constructor, então adicionamos como propriedade
Math.areaCirculo = function(raio) {
  return Math.PI * raio ** 2;
};

console.log(Math.areaCirculo(5)); // 78.54...
```

Slide 22 de 30

Prototype — Cuidados e Contexto Histórico

Quando usar e quando NÃO usar

Antes da sintaxe `class` (ES6, 2015), todo OOP em JavaScript era feito diretamente com prototypes:

A sintaxe `class` é **açúcar sintático** — por baixo, faz exatamente a mesma coisa com prototypes. Mas é muito mais legível e organizada.

Slide 23 de 30

Verificação — Prototype

Testando o entendimento

Depois de `Array.prototype.ultimo = function() { return this[this.length - 1]; }`, o que acontece?

- Apenas arrays criados DEPOIS dessa linha terão o método
- ✓ TODOS os arrays (existentes e futuros) ganham o método .ultimo() (Resposta Correta)
- Dá erro — não pode modificar classes nativas
- Só funciona com arrays de números

Explicação: Correto! Modificar o `prototype` afeta TODOS os objetos daquela classe — existentes e futuros. É por isso que é poderoso mas perigoso: uma mudança afeta o sistema inteiro.

Slide 24 de 30

Juntando Tudo — Conceitos Conectados

Como herança, polimorfismo, static e prototype se relacionam

Conceito	Palavra-chave	Para que serve
Herança	extends	Reaproveitar código de uma classe pai
super()	super	Chamar constructor ou método do pai
Polimorfismo	(sobrescrita)	Mesmo método, comportamentos diferentes
static	static	Membros que pertencem à classe, não à instância
Prototype	.prototype	Cadeia interna de herança — onde os métodos vivem

Slide 25 de 30

Verificação Geral — Aula 4

Revisão dos 4 conceitos

Qual alternativa descreve CORRETAMENTE a relação entre os conceitos?

- Polimorfismo é um tipo de herança, e static é um tipo de prototype
- ✓ Herança permite reaproveitar código; polimorfismo permite que cada filho tenha seu comportamento; static pertence à classe; prototype é o mecanismo interno de herança (Resposta Correta)
- Todos são sinônimos para a mesma coisa: reutilização de código
- Herança e polimorfismo são para instâncias; static e prototype são para funções

Explicação: Exato! Cada conceito tem seu papel: herança reaproveita, polimorfismo diferencia, static compartilha na classe, prototype é o motor por baixo.

Slide 26 de 30

Atividade Principal 1 — Catálogo/Cardápio Digital

Projeto de escopo aberto usando todos os conceitos

Atividade prática (escopo aberto): Criem um catálogo ou cardápio digital que utilize TODOS os conceitos vistos até agora.

1. **Modelar**: desenhar UML simplificado antes de codar (obrigatório!)
2. **Herança**: pelo menos 2 classes filhas que herdam de uma classe pai
3. **Encapsulamento**: campos privados com getters/setters validados
4. **Polimorfismo**: pelo menos 1 método sobrescrito nas filhas (ex: `exibir()`)
5. **Prototype**: pelo menos 1 método adicionado via prototype (ex: `Array.prototype.somarPrecos`)
6. **CRUD**: criar, listar, editar, deletar itens
7. **EXTRA**: HTML com formulário de cadastro e exibição visual do catálogo

---

Slide 27 de 30

Exemplo de Partida — Catálogo

Código inicial para ajudar a começar

```
class ItemCardapio {
  #nome;
  #preco;

  constructor(nome, preco, descricao = "") {
    this.nome = nome;
    this.preco = preco;
    this.descricao = descricao;
  }

  get nome() { return this.#nome; }
  set nome(v) {
    if (!v || !v.trim()) throw new Error("Nome inválido");
    this.#nome = v.trim();
  }

  get preco() { return this.#preco; }
  set preco(v) {
    if (typeof v !== "number" || v < 0) throw new
Error("Preço inválido");
    this.#preco = v;
  }

  exibir() {
    return `${this.#nome} – R$${this.#preco.toFixed(2)}`;
  }
}

class Bebida extends ItemCardapio {
  constructor(nome, preco, ml, temperatura = "gelada") {
    super(nome, preco);
    this.ml = ml;
    this.temperatura = temperatura;
  }
```

```
  exibir() {
    return `🥤 ${super.exibir()}
(${this.ml}ml, ${this.temperatura})`;
  }
}

// Prototype extra:
Array.prototype.somarPrecos = function() {
  return this.reduce((acc, item) => acc + item.preco, 0);
};
```

Slide 28 de 30

Lição de Casa — Aula 4

Finalizar o catálogo/cardápio

Para entregar: O projeto do catálogo/cardápio completo e funcional.

- **UML**: diagrama das classes com relações de herança
- **Herança**: pelo menos 2 classes filhas com `extends` e `super`
- **Encapsulamento**: campos privados, getters/setters com validação
- **Polimorfismo**: pelo menos 1 método sobrescrito (ex: `exibir()`)
- **Prototype**: pelo menos 1 método adicionado via prototype
- **CRUD**: criar, listar, editar, deletar itens — tudo funcionando
- **EXTRA**: HTML com formulário e exibição visual

Slide 29 de 30

Resumo da Aula 4

Revisão ativa com flashcards

Tente responder antes de virar cada cartão: